Sokaban results

With 1.5B and beam width as 10

Solver	Solved	Success%	Avg Steps	Avg LLM Calls	Avg Time
bfs	8/10	80.0%	54.5	0	0.13s
beam_w10	2/10	20.0%	23.0	70	19.81s
astar	8/10	80.0%	54.5	83	22.07s

With 3B and beam width as 25

Solver	Solved	Success%	Avg Steps	Avg LLM Calls	Avg Time
bfs	8/10	80.0%	54.5	0	0.13s
beam_w25	4/10	40.0%	44.5	169	75.13s
astar	7/10	70.0%	47.0	89	39.40s

With 1.5B and beam width as 25

Solver	Solved	Success%	Avg Steps	Avg LLM Calls	Avg Time
bfs	8/10	80.0%	54.5	0	0.12s
beam_w25	3/10	30.0%	26.3	268	70.47s
astar	7/10	70.0%	47.0	89	23.66s

With 1.5B and beam width as 25 and MCTS

Solver	Solved	Success%	Avg Steps	Avg LLM Calls	Avg Time
bfs	8/10	80.0%	54.5	0	0.13s
beam_w25	3/10	30.0%	26.3	268	70.66s
astar	7/10	70.0%	47.0	89	23.62s
mcts	2/10	20.0%	33.5	279	73.54s

With Claude sonet 4.6 not using beam search

Solver	Solved	Success%	Avg Steps	Avg LLM Calls	Avg Time
bfs	8/10	80.0%	54.5	0	0.13s
astar	8/10	80.0%	54.5	100	168.84s

only bfs

```
--------------------------------------------------------------------------------------------
Solver                  |  Solved |  Success% |  Avg Steps |  Avg LLM Calls |  Avg Time
--------------------------------------------------------------------------------------------
bfs                     |  83/150 |     55.3% |       63.7 |              0 |     0.23s
--------------------------------------------------------------------------------------------
```

Strategy	Time for 5 states	LLM Calls	Per-call Time
Normal MLX	1.283s	5 calls	~0.256s each
Cached MLX	**0.169-0.173s**	5 calls	**~0.17s each (34% faster)**
Batch	**0.358s**	**1 call**	**0.358s total (3.6x faster)**

Three Inference Optimization Strategies Implemented

1. MLX Prompt Caching (mlx_cached **backend)**
The system prompt + Sokoban rules are identical for every LLM call (~150 tokens). The new mlx_cached backend uses MLX's generate_step with prompt_cache to process these prefix tokens only once, then reuses the KV cache for subsequent calls.
Result: 34% faster per call (0.256s → 0.169s)

2. Batch Prediction (predict_batch()**)**
Instead of making N individual LLM calls for N beam states, the new predict_batch() packs multiple board states into one prompt with numbered labels, makes a single LLM call, and parses all rankings from the response.
Result: 3.6x faster for 5 states (1.283s → 0.358s, 5 calls → 1 call)

3. Batch-Aware Beam Search
BeamSearchSolver.solve() now batches all beam states into one predict_batch() call per step, instead of individual predict_top_k() calls. With beam_width=25, this means ~1 LLM call per depth step instead of ~25.
Combined impact for beam search: With 25 beam states × 200 depth steps, the old approach made up to 5000 separate LLM calls. The new approach makes ~~200 calls (one per step). That's a **25x reduction** in LLM calls, which with the prompt caching speedup translates to roughly **~~30x faster** overall inference for beam search.

Here's the honest breakdown:

Optimization	Speed Gain	Accuracy Impact	Your Situation
Prompt caching (mlx_cached)	~34% faster per call	Zero loss	✅ Use it
State-level caching	Instant for repeat states	Zero loss	✅ Already have it
Batch prediction	N calls → 1 call	**Worse for small models**	❌ Don't use with 1.5B
Only call on box pushes	~60-70% fewer calls	Zero loss (walk moves are uninformative)	✅ Already have it in A*

tried with different representations but it gave same results;ts

Using beam search solver for all 3 representations

Solver	Solved	Success%	Avg Steps	Avg LLM Calls	Avg Time
ascii	1/5	20.0%	16.0	150	38.99s
structured	1/5	20.0%	16.0	151	47.36s
annotated	1/5	20.0%	16.0	146	44.27s

this is looping across all the valid actions llm prediction is just used in priority only with groq as backed without vllm

Solver	Solved	Success%	Avg Steps	Avg LLM Calls	Avg Time
bfs	8/10	80.0%	54.5	0	0.13s
astar	8/10	80.0%	54.5	100	1000.11s

Like :
  Loop over ALL valid actions:
    action=up → push with priority = g - h - 0.0 (LLM didn't predict it)
    action=down → push with priority = g - h - 0.18 (LLM gave 0.6 score)
    action=right → push with priority = g - h - 0.30 (LLM gave 1.0 score)
    action=left → push with priority = g - h - 0.0 (LLM didn't predict it)

With groq api (facing the rate limit error issue)

  One random puzzle check (idx 13, before scaling)

  - bfs: solved, 51 steps, 0.00s
  - beam: failed, 1 LLM call, 0.49s
  - astar: solved, 51 steps, 35 LLM calls, 31.81s
  - mcts: failed, 7 LLM calls, 16.47s

  5-puzzle comparison (varying difficulty)
  Puzzles used: 55, 66, 26, 49, 62 (BFS steps: 23, 37, 50, 76, 101)

  Settings:

  - beam: beam_width=2, max_depth=60, max_llm_calls=40, cache_mode='exact'
  - astar: max_states=1200, llm_weight=0.3, max_llm_calls=40, cache_mode='macro'
  - mcts: n_iterations=220, rollout_depth=24, max_llm_calls=40, cache_mode='macro'
  - backend: Groq (fallback available; not needed in this run)

Solver	Solved	Success%	Avg Steps	Avg LLM Calls	Avg Time	Avg Cache Hits	Avg Fallbacks
bfs	5/5	100.0%	57.4	0.0	0.01s	0.0	0.0
beam	0/5	0.0%	N/A	25.6	71.98s	4.8	0.0
astar	2/5	40.0%	30.0	35.2	94.83s	428.0	0.0
mcts	0/5	0.0%	N/A	9.2	22.64s	204.4	0.0

Practical conclusion

- For your strict LLM-guided setting, A* is currently the best accuracy/robustness among LLM solvers.
- Beam is still collapsing on harder puzzles; MCTS is faster but not solving enough.
- Biggest latency driver is still remote API inference time, not just beam width.
- Most practical next move for faster + better: run the same A* pipeline on local inference (MLX/Ollama GPU) so you keep behavior but cut network latency.

Logs

Show Errors Only | Download

27/04/2026, 23:37:10	llama-3.1-8b-instant	sokaban_llm	200	0.024	0.051	198	26	–	req_0...8q20	–
27/04/2026, 23:35:52	llama-3.1-8b-instant	sokaban_llm	200	0.027	0.056	217	26	–	req_0...tqc8	–
27/04/2026, 23:35:50	llama-3.1-8b-instant	sokaban_llm	429	0	0.001	0	0	–	req_0...n3cz	rate_limit_exceeded
27/04/2026, 23:35:49	llama-3.1-8b-instant	sokaban_llm	200	0.022	0.047	217	26	–	req_0...jgvd	–
27/04/2026, 23:35:47	llama-3.1-8b-instant	sokaban_llm	429	0	0.001	0	0	–	req_0...mnb0	rate_limit_exceeded
27/04/2026, 23:35:47	llama-3.1-8b-instant	sokaban_llm	200	0.191	0.227	216	26	–	req_0...b1xh	–
27/04/2026, 23:35:44	llama-3.1-8b-instant	sokaban_llm	429	0	0.001	0	0	–	req_0...hxtz	rate_limit_exceeded
27/04/2026, 23:35:44	llama-3.1-8b-instant	sokaban_llm	200	0.194	0.219	217	26	–	req_0...27g4	–
27/04/2026, 23:35:42	llama-3.1-8b-instant	sokaban_llm	429	0	0.001	0	0	–	req_0...pa9g	rate_limit_exceeded
27/04/2026, 23:35:42	llama-3.1-8b-instant	sokaban_llm	200	0.022	0.046	216	26	–	req_0...tf7s	–
27/04/2026, 23:35:40	llama-3.1-8b-instant	sokaban_llm	429	0	0.001	0	0	–	req_0...t80x	rate_limit_exceeded
27/04/2026, 23:35:39	llama-3.1-8b-instant	sokaban_llm	200	0.024	0.048	217	26	–	req_0...b20h	–
27/04/2026, 23:35:37	llama-3.1-8b-instant	sokaban_llm	429	0	0.001	0	0	–	req_0...kjg6	rate_limit_exceeded

so using vllm as backend with amd mi3000x 192gb ram using model qwen2.5-7B-instruct

NOTE: bfs = pure BFS, zero LLM calls (baseline)
astar_vllm = A* where EVERY state is LLM-guided (Qwen2.5-7B via vLLM)
llm_calls/puzzle shows LLM involvement in solving

Solver	Solved	Success%	Avg Steps	Avg LLM Calls	Fallbacks	Avg Time
bfs (no LLM)	8/10	80.0%	54.5	0	0.0	0.13s
astar_vllm	8/10	80.0%	54.5	9585	0.0	219.08s

—> If u can see the avg time reduced from using groq as backend to vllm as backend the time is 1000 seconds which is too high but now its reduced to 200 seconds

—> if max_states and max_llm calls are reduced to 10000 and also reducing max tokes from 20 to 8 and batch size 32 to 64 reduces the avg time to 32 seconds which is fast .

Solver	Solved	Success%	Avg Steps	Avg LLM Calls	Fallbacks	Avg Time
bfs (no LLM)	9/10	90.0%	51.9	0	0.0	0.08s
astar_vllm	7/10	70.0%	34.3	4448	0.0	32.68s

this comparison is with different representations with 20k states and 20k llm calls as u can see all the representations have same accuracy and all fail for puzzle 5 and puzzle 110 so , representations truly doesn't matter

COMPARISON SUMMARY

Representation	Solved	Accuracy	Avg Steps	Avg States	Avg LLM	Avg Fallbacks	Avg Time
ASCII (raw grid)	8/10	80.0%	45.0	6594	6594	0.0	68.23s
Structured (text description)	8/10	80.0%	45.0	6608	6608	0.0	63.30s
Annotated (grid + distance hints)	8/10	80.0%	45.0	6592	6592	0.0	67.61s

Representation: ASCII (raw grid)

Puzzle	Boxes	Solved	Steps	States	LLM Calls	Fallbacks	Time
0	2	YES	33	184	184	0	3.26s
1	3	YES	16	572	572	0	5.68s
5	3	NO	—	20001	20001	0	191.85s
10	2	YES	78	2256	2256	0	21.19s
20	2	YES	17	175	175	0	1.93s
30	3	YES	17	1008	1008	0	9.55s
50	2	YES	34	505	505	0	5.26s
70	2	YES	120	11601	11601	0	108.50s
90	4	YES	45	9628	9628	0	97.41s
110	6	NO	—	20014	20014	0	237.65s

Representation: Structured (text description)

Puzzle	Boxes	Solved	Steps	States	LLM Calls	Fallbacks	Time
0	2	YES	33	184	184	0	2.80s
1	3	YES	16	588	588	0	5.58s
5	3	NO	—	20001	20001	0	180.41s
10	2	YES	78	2252	2252	0	19.85s
20	2	YES	17	175	175	0	1.85s
30	3	YES	17	1008	1008	0	9.05s
50	2	YES	34	505	505	0	5.02s
70	2	YES	120	11601	11601	0	98.77s
90	4	YES	45	9756	9756	0	91.83s

110	6	NO	—	20014	20014	0	217.82s

Representation: Annotated (grid + distance hints)

Puzzle	Boxes	Solved	Steps	States	LLM Calls	Fallbacks	Time
0	2	YES	33	184	184	0	2.86s
1	3	YES	16	588	588	0	5.77s
5	3	NO	—	20001	20001	0	190.52s
10	2	YES	78	2245	2245	0	21.08s
20	2	YES	17	175	175	0	1.92s
30	3	YES	17	1008	1008	0	9.29s
50	2	YES	34	505	505	0	5.21s
70	2	YES	120	11601	11601	0	108.32s
90	4	YES	45	9596	9596	0	95.00s
110	6	NO	—	20014	20014	0	236.10s